AI 时代的形式化验证：以 ζ 函数为例

本篇以 Lean 4 及其数学库 mathlib 中关于 ζ 函数的定理为例，说明“经过形式化验证”保证什么、不保证什么，以及在 AI 大量生成证明的情况下应当怎样阅读这类声明。文中的 Lean 代码取自本仓库 `命题的形式化验证-Lean4/`，均已由 Lean 检查；形式化验证的一般原理见学习笔记第 16 篇，此处不重复。所涉近期事件截至 2026 年 10 月下旬，均注明出处。

1 问题

传统上，一个证明是否可信，取决于同行能否读懂并逐步核对。当证明由 AI 生成、篇幅达到数百页、而人类读者有限时，这一方式遇到了瓶颈。形式化验证提供另一条途径：把命题和证明写成证明助手能够机械检查的形式，由一个很小的程序（内核）判断每一步推理是否合法。

近来越来越多的数学声明附有“已在 Lean 中形式化”的说明。本篇讨论三个问题：

1. 一条形式化陈述怎样读，它与通常的数学陈述有哪些差别；
2. Lean 接受一个证明，究竟保证了什么；
3. 证明者不可信（例如未经审查的 AI）时，还需要哪些额外的检查。

2 一条形式化陈述怎样读

mathlib 中黎曼猜想的定义如下：

```
def RiemannHypothesis : Prop :=
  ∀ (s : ℂ) (_ : riemannZeta s = 0) (_ : ¬∃ n : ℕ, s = -2 * (n + 1))
    (_ : s ≠ 1), s.re = 1 / 2
```

`Prop` 表示这是一个命题，而不是已经证明的定理。逐项读出：对一切复数 s，若 $\zeta(s)=0$，s 不是 $-2,-4,-6,\dots$ 中的某一个，且 $s\neq 1$，则 $\operatorname{Re} s=\frac{1}{2}$。

这条陈述有三处值得注意。

1. **类型**。`s : ℂ` 指明 s 是复数；`n : ℕ` 指明 n 是自然数（含 0），因此 `-2 * (n + 1)` 恰好取遍 $-2,-4,\dots$。若误写成 `-2 * n`，就会把 $s=0$ 也排除在外（因 $\zeta(0)=-\frac{1}{2}\neq 0$，这碰巧不改变命题的真假）。形式化陈述中的每个符号都有确定的类型，这消除了自然语言中的许多歧义，也带来了新的出错方式。
2. **定义的依赖**。陈述中的 `riemannZeta` 是 mathlib 中的一个定义，经由 Hurwitz ζ 函数、Mellin 变换与 θ 函数逐层构造，最终依赖实数与复数的构造。这条陈述是否“就是”黎曼猜想，取决于这一长串定义是否与数学上的 ζ 一致。mathlib 为此证明了大量性质（$\operatorname{Re} s>1$ 时等于 $\sum n^{-s}$、函数方程、Euler 乘积等），作为“定义正确”的旁证。
3. **多余的条件**。条件 $s\neq 1$ 在数学上多余，因为 ζ 在 $s=1$ 无定义。在 mathlib 中它同样多余（见下节），但写上它，陈述就不依赖于 $s=1$ 处的约定。

3 处处有定义的函数

3.1 $\zeta(1)$ 的约定值

Lean 中的函数必须在定义域的每一点取值，`riemannZeta` 的定义域是整个 $\mathbb{C}$，因此 `riemannZeta 1` 必须是某个复数。mathlib 的构造方式（其中除以零的结果规定为零）使

$$\texttt{riemannZeta 1} = \frac{\gamma - \log(4\pi)}{2} \approx -0.9769. \tag{3.1}$$

命题 3.1. *设 $\Lambda(s) = \pi^{-s/2}\Gamma(s/2)\zeta(s)$ 为完备 ζ 函数，则它在 $s = 1$ 处 Laurent 展开的常数项等于 $(\gamma - \log 4\pi)/2$。*

证明. 在 $s = 1$ 附近，$\zeta(s) = \dfrac{1}{s-1} + \gamma + O(s-1)$。又 $g(s) = \pi^{-s/2}\Gamma(s/2)$ 在 $s = 1$ 处解析，$g(1) = \pi^{-1/2}\Gamma(\frac{1}{2}) = 1$，

$$\frac{g'(1)}{g(1)} = -\frac{1}{2}\log\pi + \frac{1}{2}\,\psi\!\left(\tfrac{1}{2}\right) = -\frac{1}{2}\log\pi - \frac{\gamma}{2} - \log 2,$$

其中用到 $\psi(\frac{1}{2}) = -\gamma - 2\log 2$。于是 $g(s) = 1 + g'(1)(s-1) + O\big((s-1)^2\big)$，

$$\Lambda(s) = \frac{1}{s-1} + \gamma + g'(1) + O(s-1) = \frac{1}{s-1} + \frac{\gamma - \log 4\pi}{2} + O(s-1). \qquad \square$$

这个约定值并非任意：它是 Λ 去掉极点部分后在 $s = 1$ 处的值，也等于 $\lambda_1 - 1$，其中 $\lambda_1 = 1 + \frac{\gamma}{2} - \frac{1}{2}\log 4\pi$ 是 Li 判据的第一个系数（学习笔记第 10 篇）。但它不是 $\zeta(s) - \dfrac{1}{s-1}$ 在 $s = 1$ 处的极限 γ。两种“自然”的补充定义给出不同的值，正说明约定值不携带关于极点的数学信息。

由此，mathlib 的定理

```
theorem riemannZeta_ne_zero_of_one_le_re {s : ℂ} (hs : 1 ≤ s.re) :
    riemannZeta s ≠ 0
```

在 $s = 1$ 处断言的是 $(\gamma - \log 4\pi)/2 \neq 0$，与 ζ 在 $s = 1$ 有极点这一事实无关。它在 $s \neq 1$ 处的内容才是本书第 9 章的定理。

3.2 Γ 在极点处的值与函数方程的条件

同样，mathlib 规定 `Complex.Gamma` 在 $0, -1, -2, \ldots$ 处取 0。函数方程在 mathlib 中写作

```
theorem riemannZeta_one_sub {s : ℂ} (hs : ∀ n : ℕ, s ≠ -n) (hs' : s ≠ 1) :
    riemannZeta (1 - s) =
      2 * (2 * π) ^ (-s) * Gamma s * cos (π * s / 2) * riemannZeta s
```

条件 `hs` 排除 $s = 0, -1, -2, \ldots$。它在 Lean 中是必需的，而不只是习惯：取 $s = 0$，左边是约定值 (3.1)，不为零；右边含因子 `Gamma 0`，按约定为 0，于是右边为 0。去掉 `hs`，这条陈述在 Lean 中是假的。数学上 $s = 0$ 处右边是“极点乘零点”的形式，公式只能通过取极限理解；Lean 不做这种理解，条件必须写明。

这类约定是形式化数学的常规做法：让函数处处有定义，代价是在每条定理中写清适用范围。读者遇到“mathlib 中某公式对一切 s 成立”时，应先查明它在例外点上依赖哪些约定。

4 Lean 接受一个证明，保证了什么

4.1 保证的部分

Lean 接受一个定理，意味着内核逐步检查了一个证明项，确认它在 Lean 的逻辑中推出了所写的陈述。可信度依赖于：内核与逻辑的可靠性；所用的公理；所依赖的库定义（见学习笔记第 16 篇）。其中公理可以直接查询：命令 `#print axioms` 列出一个定理直接或间接依赖的全部公理。标准情形只有 `propext`、`Classical.choice`、`Quot.sound` 三条；若出现 `sorryAx`，说明证明中有未完成的部分；若出现其他名字，则是另外引入的公理，定理只在该公理成立时才成立。

4.2 不保证的部分：陈述

Lean 不判断陈述是否就是想要的命题。下面这条定理被 Lean 接受（本仓库 `命题的形式化验证-Lean4/BookExamples/S5_Checking.lean`）：

```
example (s : ℂ) (h1 : 1 < s.re) (h2 : riemannZeta s = 0) :
    s.re = 1 / 2 :=
  absurd h2 (riemannZeta_ne_zero_of_one_le_re h1.le)
```

它的字面意思是“若 $\mathrm{Re}\, s > 1$ 且 $\zeta(s) = 0$，则 $\mathrm{Re}\, s = \frac{1}{2}$”，结论与 RH 相同，证明只有一行。原因是两个假设不能同时成立，而从矛盾的假设可以推出任何结论。这是形式化中最典型的一类陷阱：*假设过强*，以致定理空洞成立。与之相对的是*结论过弱*，或者把定义写错，使陈述说的是另一件事。

因此，一条“已经形式化的定理”是否有数学价值，要由人回答两个问题：形式化陈述是否忠实于原意；它所依赖的定义是否正确。Lean 把需要人工审查的部分从整个证明缩小到陈述与定义，但没有消除这一部分。

5 证明者不可信的情形

Lean 的官方参考手册在“验证证明”一章中区分两种情形：*诚实*的证明尝试可能有错误，但不试图欺骗系统；*恶意*的尝试则可能利用系统漏洞，或用元程序在检查之前做手脚。手册明确把未经审查的 AI 生成证明归入后一类，并给出由弱到强的几层检查：

1. 编辑器显示通过，或 `lake build` 无错误。这只说明当前文件被接受，不能发现依赖中的 `sorry`。
2. `#print axioms`，拒绝 `sorryAx` 与非标准公理。
3. 用 `lean4checker` 把编译结果重新送入内核检查，排除某些绕过内核的漏洞。
4. 最强的做法是 `comparator`：由可信的一方单独写出*挑战文件*，其中只有定理陈述而没有证明；待检查的证明在隔离环境中编译并导出，再在隔离环境之外由 Lean 内核或独立实现的外部检查器重新检查，并逐字比对它证明的陈述是否与挑战文件一致。

第 4 层的要点在于分工：证明可以来自任何人或任何程序，而陈述由可信的人写定。审查者不必阅读证明，只需审查挑战文件中的陈述及其所用定义是否正确。这正对应上一节所说的“需要人工审查的部分”。

Tao 在评论 AI 完成形式化项目时也指出，项目的组织者今后需要把全部目标明确写出，因为 AI 只会优化被明确陈述的目标。形式化把“证明是否正确”交给机器，同时把“陈述是否正确”的责任更集中地留给人。

6 两个实例

6.1 强形式素数定理（2025）

2024 年 1 月，Tao 与 Kontorovich 发起在 Lean 中形式化素数定理（带经典误差项）及相关结果的项目 PrimeNumberTheoremAnd。到 2025 年中，项目取得部分进展，主要困难在于复分析基础结果的缺失。2025 年 9 月，Math Inc. 公司报道其自动形式化代理 Gauss 用约三周完成了强形式素数定理的形式化，代码约 25 000 行，期间补齐了所需的复分析结果。（在此之前，素数定理已于 2007 年在 Isabelle 中、2009 年在 HOL Light 中形式化，分别采用初等证明与解析证明。）

这一例子说明，AI 在形式化中的作用之一是承担大量“已知数学”的翻译工作。陈述本身（素数定理）没有争议，验证的价值在于证明的完整性。

6.2 “拟黎曼猜想”（2026）

2026 年 10 月 6 日，OpenAI 公开一批由其内部模型生成的数学稿件（据报道共 722 份），其中据报道约四到六成附有 Lean 证明。其中一份 199 页的稿件声称：ζ 与一切 Dirichlet L 函数在半平面 $\operatorname{Re} s > \frac{7}{8}$ 中无零点（$s = 1$ 处的极点除外），并注明主要结果已在 Lean 中形式化，附有 comparator 检查的链接；稿件的后续应用部分未形式化。另一份 49 页的稿件以相近方法给出 $\frac{11}{12}$ 的结论，未作形式化。据 *Quanta Magazine* 10 月 19 日的报道，专家大体认为 $\frac{11}{12}$ 的结果正确，已有人能用很短的篇幅概述其论证；也有专家表示，在人工核对全部细节之前不视其为定理，并认为新方法未必使 RH 本身更近。上述稿件均未经同行评审。本文未核查其 Lean 文件，以下只讨论这类声明应当怎样阅读。

若 ζ 在 $\operatorname{Re} s > \theta$ 中无零点（$\theta < 1$），由显式公式即得 $\psi(x) = x + O(x^{\theta} \log^2 x)$；这类结论统称拟黎曼猜想，此前没有任何 $\theta < 1$ 被证明。依照前两节，对“已在 Lean 中形式化”的声明，读者可以依次追问：

1. 挑战文件中的陈述是什么，是否就是稿件所声称的结论；例如例外点 $s = 1$ 怎样处理，L 函数用的是哪个定义。
2. 证明依赖哪些公理，有无 `sorry`，是否经独立检查器复核。
3. 形式化覆盖了稿件的哪些部分。

前两项若都得到肯定的回答，结论的正确性就有了很强的保证，而且这种保证不依赖于有人读完 199 页。但形式化回答不了另外的问题：证明的思想是什么，能否推广，是否新颖，结论归功于谁。这些问题仍要靠人阅读和理解；上述报道中数学家的工作，主要也在于此。

7 本书内容的形式化现状

下表列出本书主线上若干结论在 mathlib 中的情况（以本仓库所用的 mathlib 版本为准，2025 年 8 月）。

内容	书中位置	情况
Euler 乘积	第 5 章	mathlib 已有
Γ 的倍元公式、余元公式	第 7 章	mathlib 已有
ζ 的延拓与函数方程	第 9 章	mathlib 已有
$\mathrm{Re}\, s \geq 1$ 上无零点	第 9 章	mathlib 已有
RH 的陈述；平凡零点	第 11 章	mathlib 已有（RH 未证明）
ξ 的 Hadamard 乘积	第 10 章	未收入 mathlib
零点计数公式 $N(T)$	第 11 章	未收入 mathlib
素数定理	第 12 章	未收入 mathlib；已在独立项目中完成
显式公式	第 14 章	未收入 mathlib
RH 的算术等价形式	第 15 章	未收入 mathlib

“未收入 mathlib”只说明还没有人把它形式化并收入这个库，不说明它不能形式化。从上节的例子看，这些空白可以预期会由人与 AI 合作逐步补齐，那时本书的论证原则上也可以在形式上逐条核对；而选择哪些陈述、怎样组织它们，仍然是数学写作本身的任务。

8 结论

1. 形式化陈述必须给出每个对象的类型和定义；mathlib 让函数处处有定义，代价是在每条定理中写清例外点。$\zeta(1)$ 的约定值 $(\gamma - \log 4\pi)/2$ 与函数方程的条件 `hs` 是两个具体例子。
2. Lean 接受一个证明，保证的是在其逻辑和所列公理下由假设推出结论；它不保证陈述忠实于原意，假设矛盾的定理同样会被接受。
3. 对不可信的证明者，可靠的做法是由可信的一方写定陈述（挑战文件），再用独立检查器核对证明。人工审查的对象于是从证明转移到陈述与定义。
4. 形式化能确立正确性，不能代替理解。AI 时代的数学阅读，需要同时具备读懂形式化陈述和读懂数学思想两种能力。

参考

- The Lean Language Reference, Validating Proofs. https://lean-lang.org/doc/reference/latest/ValidatingProofs/
- D. Loeffler, M. Stoll, Formalizing zeta and L-functions in Lean, *Annals of Formalized Mathematics*, 2025. https://arxiv.org/abs/2503.00959
- PrimeNumberTheoremAnd. https://github.com/AlexKontorovich/PrimeNumberTheoremAnd
- J. D. Lichtman, Gauss — an agentic formalization of the Prime Number Theorem, Fields Institute 报告，2025 年 10 月。https://www1.fields.utoronto.ca/talks/Gauss-agentic-formalization-Prime-Number-Theorem
- 量子位，2025 年 9 月 14 日关于 Gauss 完成强形式素数定理形式化的报道。https://www.qbitai.com/2025/09/331038.html

- J. Avigad, K. Donnelly, D. Gray, P. Raff, A formally verified proof of the prime number theorem, *ACM Trans. Comput. Logic* 9 (2007).
- J. Harrison, Formalizing an analytic proof of the prime number theorem, *J. Automated Reasoning* 43 (2009).
- AI proved the quasi-Riemann hypothesis. What happens now?, *Quanta Magazine*, 2026 年 10 月 19 日。https://www.quantamagazine.org/ai-proved-the-quasi-riemann-hypothesis-what-happens-now-20261019/
- 本仓库 命题的形式化验证-Lean4/；学习笔记第 10 篇（Li 判据）、第 16 篇（计算与证明）；本书第 9、11 章。